meeami-tts

Time-aligned US-English text-to-speech on the word timing of a non-US reference

Project report · Chaganti Raja Ravi Teja · November 2026

Code: https://github.com/rrtchaganti/meeami-tts
Trained models and all test outputs: https://huggingface.co/Raviteja546/meeami-tts

23.9 ms	0	2.9%	4.20 / 5
average word-timing error (original VITS: 40.5 ms)	samples of length difference on all 1,248 test sentences	words misrecognized by ASR (in the non-US recordings: 6.3%)	predicted naturalness (UTMOS) (LJSpeech recordings: 4.36)

Results of the main system on accents never used during development (Arabic, Korean, Vietnamese; 12 speakers × 104 sentences), evaluated once after the code was frozen. ASR = automatic speech recognition. All terms are explained in the glossary after the contents.

Abstract

Given an English text and a recording of it by a non-US speaker, the system speaks the text in the voice and US accent of LJSpeech (a public dataset of one US female speaker) while following the recording's timing: the output is exactly as long as the recording, and every word starts and ends where the speaker said it, pauses included. The recording contributes timing only. The approach makes word durations an *input* to the model. VITS, a text-to-speech model that produces audio directly from phones, normally learns its own timing with a method called monotonic alignment search (MAS). We fine-tune it with phone durations measured by the Montreal Forced Aligner (MFA) instead, so durations can be set from outside. At synthesis, MFA finds the word boundaries in the recording, and each word's time span is filled with the model's own natural rhythm, stretched the way the LJSpeech speaker herself varies. On 1,248 test sentences from three unseen accents, words land on average 23.9 ms from where a human annotator marked them in the recording (the aligner's own error on accented speech is 19.4 ms), against 40.5 ms for the original VITS given the same timing and 31.9 ms for FastSpeech2, another widely used model. Two defects heard in listening were traced to their cause and fixed with minimal, controlled changes; two data-augmentation experiments did not help and are reported as negative results. Every choice was made on other accents before the test accents were opened.

I used Claude (Anthropic), an AI assistant, to speed up specialised parts of the work, such as tooling, evaluation scripts, figures and documentation. The research direction, design choices, listening judgements and final decisions are mine.

Contents

Terms and abbreviations 3

1. Introduction 5

1.1 Task . . . 5

1.2 Contributions . . . 6

1.3 Constraints . . . 6

2. Data and the split by accent 6

3. Method 7

3.1 Design principle: one source for each property of the output . . . 7

3.2 Text front end . . . 7

3.3 Forced alignment and its error on accented speech . . . 8

3.4 Models . . . 8

3.5 Synthesis: transferring the word timing . . . 10

4. How the systems are evaluated 11

5. Experiments and ablations 12

5.1 Tuning the synthesis settings (development accents) . . . 12

5.2 First comparison: M1 against M0 (development accents) . . . 12

5.3 "Words end sharply" → position-in-phone embedding . . . 12

5.4 Pace augmentation, Ablations A and B (negative result) . . . 13

5.5 "cry" heard as "cra" → two-token diphthong . . . 13

5.6 The immediate baseline V0, and how faithfully each model renders its durations . . . 14

6. Results on the test accents 15

7. Efficiency 18

8. Robustness and live testing 19

9. Limitations and future work 19

10. Reproducibility and deliverables 20

Appendix A. Deviations from the plan 20

Appendix B. Claim → evidence 21

How to read this report. Every table and figure is generated from the result files in the repository by `scripts/build_report_pdf.py` (command: `make report-pdf`). References such as D-85 or E-35 point to numbered entries of the project's decision and error log, `docs/DECISIONS.md`. `docs/REPORT.md` is a longer version of this report that links every claim to the exact line of code behind it. The audio behind every claim is in `samples/`: exact copies of the files that were evaluated.

Terms and abbreviations

Every short form, model name and technical term used in this report, in plain words. Each is also explained where it first appears.

Term	Meaning
Systems compared	
V0	The original pretrained VITS model for LJSpeech, used exactly as released (it learns its timing with MAS). The immediate baseline: it is the model that M1 starts from.
M0	A pretrained FastSpeech2 model with a HiFi-GAN vocoder. The second baseline.
M1 (VITS-MFA)	VITS fine-tuned with phone durations from MFA instead of MAS, so that durations can be set from outside.
m1_cont	M1 trained 20,000 more steps with no change. The control for "more training".
m1_pos	M1 plus a position-in-phone embedding (see below).
m1_pos_diph (main)	m1_pos plus two-token diphthongs. The main system of this project.
abl_a_pos, abl_b_pos	m1_pos trained with pace augmentation A or B: the two ablations.
ablation	An experiment that adds or removes one component to measure what it contributes.
control	The system that differs from the tested one in exactly one setting, so any difference is caused by that setting.
Data and speech	
LJSpeech (LJ)	A public dataset of 13,100 clips (about 24 h) read by one US female speaker. The target voice; the only training data.
L1, L2, L2-ARCTIC	L1 = a speaker's native language; an L2 speaker speaks English as a second language. L2-ARCTIC is a public corpus of 24 L2 speakers (6 native languages, 4 speakers each) reading the same sentences, with word and phone boundaries corrected by hand for about 150 sentences per speaker.
reference	The non-US recording whose timing the output must follow.
development (dev) / test accents	Dev accents (Hindi, Mandarin, Spanish) were used for every decision. Test accents (Arabic, Korean, Vietnamese) were kept locked and evaluated once at the end, so the results show performance on unseen accents.
phone	The smallest sound unit of a word; "cry" has three: K, R, AY.
ARPAbet	The phone alphabet of CMUdict. A digit marks stress: 0 none, 1 primary, 2 secondary (AY1 = stressed "eye" vowel, AH0 = the unstressed "uh").
diphthong	A vowel that glides from one position to another: AY ("cry"), AW ("how"), EY ("may"), OW ("go"), OY ("boy").
CMUdict	The Carnegie Mellon University Pronouncing Dictionary of US English (about 134,000 words).
lexicon	This project's word → phones file (data/lexicon/lexicon.txt), the only source of pronunciations.
G2P, g2p_en	Grapheme-to-phoneme: predicting the phones of a word from its spelling. g2p_en is a neural G2P model for English.
OOV	Out of vocabulary: a word that is not in CMUdict.
IPA, espeak	International Phonetic Alphabet. espeak is an open-source speech synthesizer whose IPA output the original VITS was trained on.
frame	The time step of the model: 256 audio samples at 22,050 samples per second, i.e. 11.6 ms. Durations are counted in frames.
sil, sp, spn, pad	Special tokens: silence at the start and end of a sentence; a pause between words; a word the aligner could not recognize; padding.
mel spectrogram	A picture of audio over time and frequency, with frequencies on the mel scale (spaced like human hearing). Log-mel = its logarithm.
formants (F1, F2)	Resonance frequencies of the mouth and throat. F1 and F2 identify a vowel, and their movement shows a diphthong's glide. Not related to the F1 score of pauses.
LPC	Linear predictive coding, a standard way to measure formants from audio.
Models and methods	
TTS	Text-to-speech.
VITS	"Variational Inference with adversarial learning for end-to-end Text-to-Speech" (Kim et al., 2021): a TTS model that turns phones directly into a waveform. Its parts: a text encoder (phones → hidden states), a prior (what each frame should look like, predicted from the text), a posterior encoder (encodes real audio; used only in training), a normalizing flow (an invertible network connecting the two) and a HiFi-GAN decoder (produces the waveform).

Term	Meaning
MAS	Monotonic alignment search: VITS's built-in method that, during training, decides which audio frames belong to which input token.
forced alignment, MFA	Finding where each word and phone starts and ends in a recording, given its transcript. MFA = Montreal Forced Aligner (version 3.4.3, pretrained english_us_arpa model). Inside, it uses an HMM-GMM: a hidden Markov model with Gaussian-mixture sound models, the classic speech-recognition design.
CTC aligner, MMS_FA	A second aligner built on Meta's MMS (Massively Multilingual Speech) recognition model, trained with CTC (connectionist temporal classification). Used as a fallback when MFA fails, and as an independent second judge.
FastSpeech2	A widely used TTS model that predicts each phone's duration, pitch and energy explicitly (in its "variance adaptor"); a given duration can be passed in directly (d_targets).
HiFi-GAN, vocoder	A vocoder turns a spectrogram (or hidden features) into a waveform. HiFi-GAN is a fast neural vocoder trained as a GAN.
fine-tuning, warm start	Continuing to train an already trained model on a changed task, starting from its weights.
duration predictor	The part of the model that predicts how many frames each phone lasts.
$\hat{d}_i$, d_i, D, σ_i, z	$\hat{d}_i$: the model's natural (native) length of phone i. d_i: the length actually used. D: the word's length in the reference, in frames. σ_i: how much LJ naturally varies phone i (standard deviation of its log duration). z: one number per word saying how much slower ($z > 0$) or faster ($z < 0$) than LJ, in units of σ.
elastic fit	Filling a word's reference time span by scaling each phone by $\exp(z \cdot \sigma_i)$: phones LJ varies a lot stretch more, stable phones (like stops) barely change.
stretch cap	The most a word is stretched: 2.5× its natural length. Any extra time becomes a pause after the word.
stretch ratio	Output speech length ÷ the model's natural length for the same words. 1.0 = LJ's own pace.
noise scale	How much randomness is sampled from the VITS prior when generating (0.333 here; larger = more varied but less clear).
position-in-phone embedding	A learned vector added to every frame that tells it where it is inside its phone (16 steps from start to end).
two-token diphthong	Each diphthong is split into two model tokens (e.g. AY1 → AY1 + AY1+), so its start and end can sound different.
pace augmentation	Training on extra copies of LJ clips whose words were made slower or faster. A: random paces. B: paces sampled from the dev-accent speakers' statistics.
WSOLA, Rubber Band R3	Ways to change audio speed without changing pitch. Rubber Band R3 (a phase vocoder) was planned; WSOLA (waveform-similarity overlap-add) was used because R3 damaged the audio.
ρ (rho)	Pace of a reference: each word's length ÷ LJ's typical length for the same phones. 1.12 = 12% slower than LJ.
losses	L_mel: spectrogram error. L_KL: KL divergence, which pulls the prior toward the encoded real audio. L_dur: duration error. L_adv and L_fm: adversarial and feature-matching losses of the GAN.
GAN, discriminator	Generative adversarial network: a second network (the discriminator) learns to tell real audio from generated audio, and the generator learns to fool it.
bf16, fp32	16-bit (bfloat16) and 32-bit floating-point number formats; bf16 is faster, fp32 more precise.
U(a, b), N(μ, s^2), AR(1)	A uniform random number between a and b; a normal (Gaussian) random number with mean μ and variance s^2; a sequence where each value is correlated with the previous one.
typed-text pre-pass	Rules that rewrite typed input (money, dates, URLs, letters…) into plain words before the normalizer.
Metrics	
word-boundary error (MAE)	Mean absolute error: for every word, how far (in ms) its start and its end in the output are from the human-annotated start and end in the reference, averaged. Lower is better.
within 20 ms / 50 ms	The share of word starts and ends that are at most 20 (or 50) ms from the annotation.
length error	Output length − reference length, in samples. Must be exactly 0.
pause F1	How well the output's pauses (silences of 100 ms or more) match the reference's: the F1 score, which combines precision (output pauses that are real) and recall (real pauses that the output has). 1 = perfect.
WER, CER	Word (character) error rate: the share of words (characters) that an automatic speech recognizer gets wrong when it transcribes the audio. Measures whether the words are said clearly. Lower is better.
ASR, Whisper	Automatic speech recognition. Whisper large-v3 is OpenAI's ASR model (run through the faster-whisper implementation).

Term	Meaning
US-accent probability	The probability that a speech sample has a US accent, from an accent classifier (CommonAccent ECAPA, an ECAPA-TDNN, a time-delay neural network, trained to recognize English accents). 0 to 1.
speaker similarity	Cosine similarity (−1 to 1) between voice embeddings from an ECAPA speaker model: to LJ (should be high) and to the reference speaker (should be low).
UTMOS	An automatic predictor of how natural speech sounds, on the 1-5 MOS scale (MOS = mean opinion score, the average rating listeners would give). LJ's own recordings score 4.36.
MUSHRA	A standard listening-test format in which listeners rate several versions of the same sentence side by side.
bias, scatter (fidelity)	Comparing where a word ends up in the output with where the pipeline placed it: bias = the average offset (negative = early); scatter = the remaining average distance once that offset is removed.
Statistics	
95% CI	Confidence interval: the range that contains the true average with 95% probability. Computed by bootstrap: re-drawing the sentences at random 1,000 times and recomputing the average.
paired comparison	Two systems speak the same sentences, so the difference is computed sentence by sentence, which removes the variation between sentences.
p-value, p	The probability of seeing a difference at least this large if the two systems were really equal. Small p = the difference is unlikely to be chance. 1e-8 means 0.00000001.
Wilcoxon signed-rank test	A standard test of whether paired differences are centred on zero, without assuming they follow a normal distribution.
Holm correction	Each comparison tests 8 metrics, and with many tests some will look significant by pure chance. Holm's method raises the p-values to correct for this. "p (Holm)" is the corrected value; a result counts as significant when it is below 0.05.
significant, n.s.	Significant: corrected p below 0.05. n.s.: not significant.
median, 90th percentile, tercile	Median: half the values are below it. 90th percentile: 90% are below it. Terciles: three equal-sized groups.
Spearman correlation	A measure (−1 to 1) of whether two quantities rise and fall together; near 0 = unrelated.
Efficiency and project terms	
parameters (M)	The number of learned weights in a network, in millions.
MACs, GMACs	Multiply-accumulate operations, the basic unit of neural-network computation; GMACs = billions of them.
RTF	Real-time factor: computing time ÷ length of the audio produced. Below 1 = faster than real time.
p50 latency	The median time over repeated runs (p90: the time 90% of runs stay under).
GPU, CPU, threads, CUDA	Graphics processor (RTX 4060 laptop GPU; L40S = the rented data-center GPU used for training) and main processor; threads = CPU cores used; CUDA = NVIDIA's GPU computing platform.
git tag freeze-v1	A label on the code version (commit 5889ecf) at which the models and experiment settings were fixed. The test accents could only be read after it existed.
D-n, E-n	Numbered entries of the project's decision log (D) and error log (E) in docs/DECISIONS.md.
HF Hub, sha256	The Hugging Face Hub, where the trained models and all test outputs are hosted; sha256 = a fingerprint of a file used to check it is unchanged.
make, conda env	make runs the project's named commands (e.g. make test); a conda env is an isolated, version-pinned set of software packages.

1. Introduction

1.1 Task

Inputs: an English text, and a recording of that text read by a **non-US** speaker (the *reference*). **Output:** speech that (1) says exactly the input words with a US accent, (2) is in the LJSpeech voice and (3) follows the reference's timing: the same total length, every word starting and ending at about the reference time, pauses included. The reference's voice, accent and pronunciation must not carry over into the output.

Why it is hard. Part of an accent lies in timing: speakers of English as a second language (L2 speakers) lengthen, shorten and pause differently from native speakers. The system must copy

the timing of *words* from the reference but keep the timing of the *phones* (the individual sounds) inside each word native. It must also stay natural when a word has to be much slower or faster than the voice would normally say it.

1.2 Contributions

- **Durations as an input.** VITS fine-tuned with MFA phone durations instead of MAS (we call it VITS-MFA), so a reference recording can dictate the timing. Compared with the unmodified VITS (system V0) given the same timing, the average word-timing error falls from 40.5 to 23.9 ms.
- **Word timing transferred with a native rhythm** (the "elastic fit" with a stretch limit and exact rounding): the output is exactly as long as the reference, by construction, for every sentence.
- **Two defects diagnosed and fixed**, each from a listening observation, each with a success rule written down in advance and a control model that differs in one setting only: a position-in-phone embedding (US-accent probability +0.09 on the development accents) and two-token diphthongs ("cry" heard as "cra"; test word error rate 4.4% → 2.9%).
- **Two pace-augmentation experiments** (training on artificially slowed and sped-up speech, A: random paces, B: paces of real non-US speakers), reported as negative results.
- **An evaluation on unseen accents**: the test accents are locked in code until the end, systems are compared sentence by sentence with corrected statistics, and extra measurements separate the model's own timing error from the aligner's measurement error.

1.3 Constraints

Constraint	Consequence for the design
Train on LJSpeech only	Voice and accent come from one US speaker. The non-US recordings are used only to evaluate, to tune and for Ablation B's statistics.
Must work on accents not seen during development	The non-US corpus (L2-ARCTIC) is split by accent; the test accents are locked in code until the git tag freeze-v1 (the frozen code version) exists.
One rented data-center GPU (NVIDIA L40S) for about 8 hours, plus an 8 GB laptop GPU	The main model M1 is trained once on the whole budget; every later experiment continues from it for 20,000 training steps.
A live demonstration on unknown recordings	A suite of difficult test cases, a second aligner as fallback, output length exact by construction, and rules for typed text.

2. Data and the split by accent

Dataset	Use
LJSpeech 1.1: 13,100 clips of one US female speaker, 22,050 samples per second	Training, with the same training / validation / test lists as the original VITS (12,500 / 100 / 500), so the pretrained model never saw the validation or test clips. MFA aligned 13,073 of them (99.79%).
L2-ARCTIC v5: 24 non-US speakers, 6 native languages × 4 speakers	The reference recordings. Its hand-corrected word boundaries are the ground truth for timing.
Development (dev) accents: Hindi, Mandarin, Spanish	Tuning, every experiment decision, Ablation B's pace statistics. 100 sentences × 12 speakers = 1,200 references.
Test accents: Arabic, Korean, Vietnamese	Evaluated once, after the code was frozen. 104 sentences × 12 speakers = 1,248 references.

Table 1. Data. Only sentences with a hand annotation for every speaker of the split are evaluated.

Why split by accent and not by sentence. The claim to support is "works for accents the system was never tuned on"; splitting by sentence would let tuning hear every accent. **The lock is enforced in code:** reading a test speaker's audio raises an error unless the git tag `freeze-v1` exists, and a unit test fills fake test files with junk bytes so that any early read would fail.

3. Method

3.1 Design principle: one source for each property of the output

Output property	Its only source	Deliberately not taken from
Words	the input text, after normalization	the reference audio or a transcript of it
Pronunciation	the project lexicon (first CMUdict pronunciation; a G2P model for unknown words)	the reference's pronunciation
Voice, accent, intonation	the model, trained on LJSpeech only	the reference (no accented audio is used for training)
Word start and end times, pauses, total length	the reference, aligned with MFA	the model's own durations
Timing of phones inside a word	the model's natural durations, fitted to the word's time span	the reference's phone timing, which carries its accent

Table 2. Nothing is averaged between the reference and the model, so each kind of failure has one place to look. Evidence for the last row: copying the reference's phone durations gave the worst word error rate of all settings tried (10.6% against 2.5%; §5.1). CMUdict = Carnegie Mellon University's pronouncing dictionary; G2P = a model that predicts pronunciation from spelling.

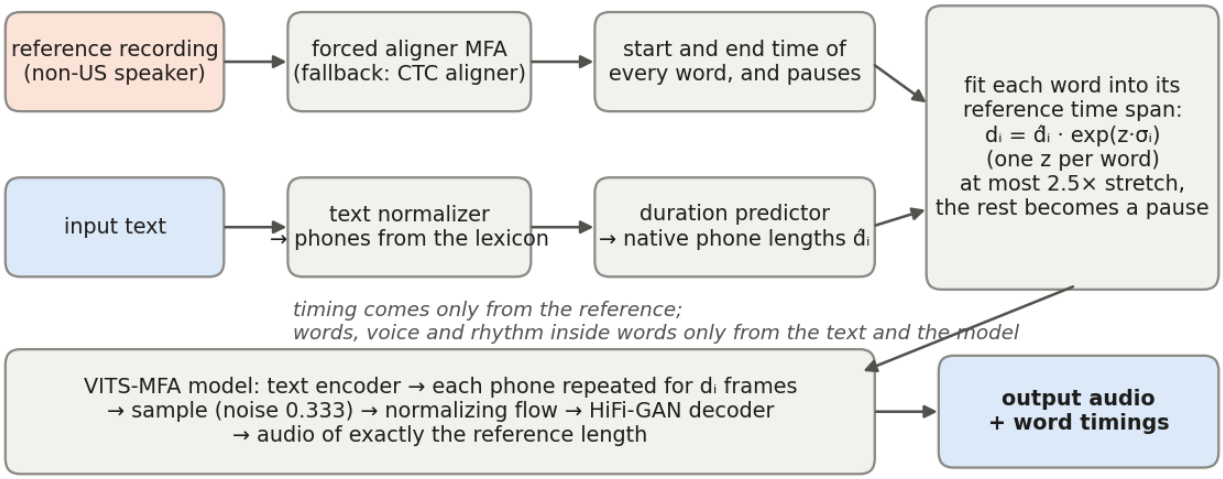


Figure 1. How an output is made. Timing enters only through the reference's word boundaries; inside each word the model's natural rhythm is stretched the way LJ herself varies (§3.5). MFA and CTC are two forced aligners (they find where each word is in a recording); $\hat{d}_i$, d_i, z and σ_i are defined in §3.5; the text encoder, normalizing flow and HiFi-GAN decoder are the parts of the VITS model (§3.4).

3.2 Text front end

- **One text normalizer** for the training text, the aligner's transcripts and the input at synthesis. It writes out currency, symbols, dates, times, abbreviations and numbers as words, in that order; any digit left over stops with an error instead of being silently dropped.
- **Lexicon:** 14,912 words with one pronunciation each, written in ARPAbet (the phone alphabet of CMUdict, e.g. "cry" = K R AY1): 13,640 from CMUdict, 1,168 predicted by g2p_en (a neural spelling-to-pronunciation model) for words not in CMUdict, and 104 possessives (*x*'*s*) built by a rule, because g2p_en mangled them. Words first seen at synthesis are predicted and appended to the same file, so it stays the only source of pronunciations.
- **Model tokens:** 69 ARPAbet phones plus three special tokens (padding, silence at the ends, a pause between words) = 72. This is exactly MFA's phone set (checked by a test), so every token has a measured duration.
- **Typed-text rules** (added after the freeze for live input): money, times, dates, ratios, units, web addresses, e-mail addresses, Roman numerals, joined words such as CamelCase, abbreviations spelled as letters, and refusals for non-Latin script. 119 of 121 test inputs come out right; a test checks that the words of every corpus sentence are unchanged.

3.3 Forced alignment and its error on accented speech

Forced alignment finds where each word and phone starts and ends in a recording whose text is known. MFA 3.4.3 (the Montreal Forced Aligner, with its pretrained US-English model and our lexicon) aligns three things with the same settings: LJSpeech (to get the training durations), the references (to get the timing to copy) and the model outputs (to measure their timing). Using one aligner everywhere means its conventions cancel out in comparisons. Its search is widened (beam 100, retry 400) and every setting is validated, because MFA silently ignores options it does not know. Boundaries are converted to frames (one frame = 11.6 ms) by rounding cumulative times, so rounding errors cannot add up; silences of at least 3 frames inside a sentence become pauses; every phone gets at least one frame.

Development accent	Word boundaries: mean error	Words within 20 ms	Phone boundaries: mean error	Phones within 20 ms
Hindi	18.2 ms	76.8%	12.3 ms	89.0%
Mandarin	20.2 ms	74.2%	14.0 ms	86.7%
Spanish	19.9 ms	74.0%	13.2 ms	87.6%
All (1,800 sentences)	**19.4 ms**	**75.0%**	**13.1 ms**	**87.7%**

Table 3. How far MFA's boundaries are from the hand-corrected boundaries of L2-ARCTIC (results/mfa_l2_dev.json). The system's timing metric aligns the output with MFA and compares it with the hand annotation of the reference, so about 19 ms of aligner error is part of every system's score: the main system's 23.9 ms is close to this floor. The fallback aligner (a CTC aligner built on Meta's MMS speech-recognition model) is less accurate, about 53 ms.

3.4 Models

VITS in brief. VITS turns a sequence of phones directly into a waveform. A *text encoder* turns the phones into hidden states; each phone's state is repeated for as many frames as the phone lasts, giving the *prior* (what each frame should sound like); a *normalizing flow* (an invertible network) maps samples from the prior to acoustic features, and a *HiFi-GAN decoder* turns these into audio. In training, a *posterior encoder* encodes the real audio, and the model learns which frames belong to which phone with *MAS* (monotonic alignment search), searched again at every training step.

V0, the immediate baseline (used as is, not retrained): the official pretrained LJSpeech VITS exactly as released, with MAS, its random (stochastic) duration predictor and its input of IPA phonetic characters with a blank token between each pair. It is the model M1 is fine-tuned from, so comparing V0 with M1 measures the effect of replacing MAS. To give it the same pronunciations, the lexicon's phones are written in the IPA characters it was trained on; its own predicted durations give each phone's natural length, and the same timing fit drives it. Its one adapter choice (which phone a blank token belongs to) was made on the development accents (blank → next character: 40.7 ms against 50.8 ms).

M0, the second baseline (used as is): a pretrained FastSpeech2 model with a HiFi-GAN vocoder. FastSpeech2 predicts each phone's duration explicitly, so the reference durations can be passed straight in.

M1, VITS-MFA: VITS fine-tuned from the official checkpoint (553 matching weight tensors loaded) with six changes:

#	Change	Reason
1	Remove MAS; frames are assigned to phones from MFA's durations	Durations must be settable at synthesis. MAS learns its own alignment inside the model, which a reference cannot override.
2	No blank tokens between phones	MFA measures no duration for blanks.
3	72 ARPAbet tokens instead of 178 IPA characters; the token embedding starts fresh	The input alphabet changed.
4	A deterministic duration predictor, trained on the log of MFA's frame counts	Only the natural relative lengths inside a word are needed; random sampling would only add noise.

#	Change	Reason
5	Optional position-in-phone embedding (16 steps from start to end)	Lets a frame's target change within a phone (§5.3).
6	Data loading from the project's file lists	One format for all data.

Table 4. The six changes from VITS to VITS-MFA. The posterior encoder, flow, HiFi-GAN decoder and the training losses are unchanged.

$$L_G = 45{\cdot}L_{mel} + L_{KL} + L_{dur} + L_{adv} + 2{\cdot}L_{fm},\ L_{dur} = mean_i\ (\log \hat{d}_i - \log d_i)^2$$

The total training loss L_G adds: the spectrogram error L_{mel}; the KL divergence L_{KL}, which pulls the prior toward the encoded real audio; the duration error L_{dur}; and the two GAN losses (L_{adv}: fool a discriminator network that tells real from generated audio; L_{fm}: match the discriminator's internal features). In L_{dur}, $\hat{d}_i$ is the predicted and d_i the measured length of phone i; taking logs makes the error relative (being 2× off costs the same for a short and a long phone). **Training:** 114,000 steps of 32 clips on one L40S GPU, about 8 hours, with 16-bit arithmetic (bf16) for the networks and 32-bit (fp32) for the losses. Later variants continue from M1 for 20,000 steps each with a fresh optimizer; new parts start at values that leave the model unchanged, so every variant *is* M1 at its first step.



Figure 2. Training of M1. The validation duration error falls throughout. The validation spectrogram error is lowest near 12,000 steps and then rises slightly while the GAN losses trade off: training and validation drift together and the discriminator loss stays level, which indicates a balanced GAN rather than overfitting. Faint lines: raw values; solid: average over 25 logged points. Validation = 100 held-out LJSpeech clips.

System	Difference from M1 (one setting each; a test checks this)
m1_cont	20,000 more steps, no change: the control for "more training"
m1_pos	position-in-phone embedding switched on (it starts at zero, so the model starts as M1)
abl_a_pos	m1_pos + 40% of each batch from pace augmentation A (random paces)
abl_b_pos	m1_pos + 40% of each batch from pace augmentation B (paces of the development-accent speakers)
m1_pos_diph (main)	**m1_pos + two-token diphthongs: each diphthong X becomes X and X+ (87 tokens)**

Table 5. The later variants. Same starting point, same number of steps, same fresh optimizer; each pair differs in one setting.

What a fixed external alignment takes away. With MAS, the model keeps adjusting which frames belong to which phone, using its own view of the sound. MFA's alignment is fixed before training by a classic statistical model (an HMM-GMM: hidden Markov model with Gaussian

mixtures) that never sees the network's internal representation. The learning signal still reaches every part of the network, but that feedback is lost: boundaries MFA places wrongly stay wrong for all of training, and, without blank tokens, all frames of a phone share one target. Measured on LJSpeech, every MFA-trained model has 14-16% less sound change inside phones than the real recordings (Figure 3). What it buys is control: these models put words where they are told to (§5.6).

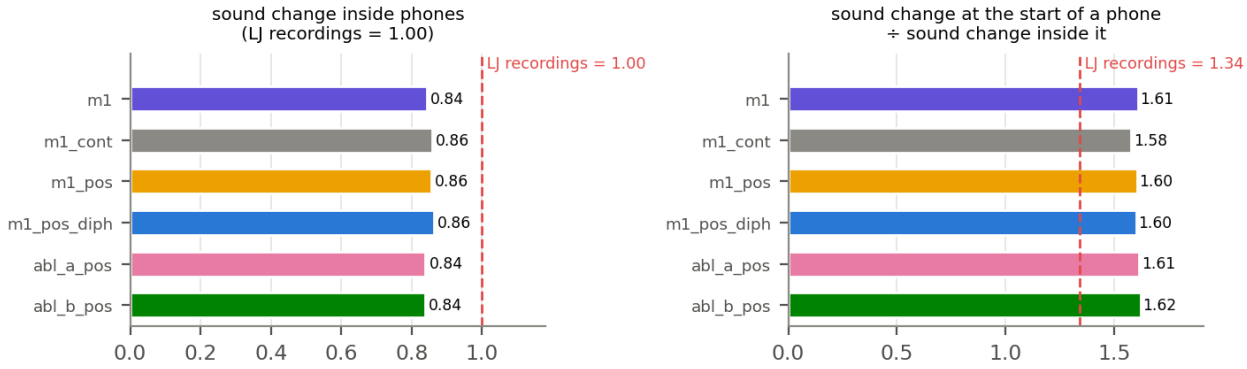


Figure 3. Sound change within phones, measured on 100 LJSpeech validation clips re-synthesized with their measured durations: the average change between consecutive frames of the log-mel spectrogram (a picture of the sound over time and frequency, on a hearing-like frequency scale). Left: inside phones, relative to the real recordings. Right: change at the start of a phone divided by change inside it; the models pile their change up at phone starts.

3.5 Synthesis: transferring the word timing

For a word that lasts D frames in the reference, the duration predictor gives a natural length $\hat{d}_i$ for each of its phones. All phones of the word are scaled with a single number z, found by bisection (repeated halving of an interval) so that the phones exactly fill the word:

$$d_i = exp(log\ \hat{d}_i + z \cdot \sigma_i),\ \Sigma_i\ d_i = D$$

σ_i is how much LJ naturally varies phone i (the standard deviation of its log duration in LJSpeech), so z says how much slower ($z > 0$) or faster ($z < 0$) than LJ the word is, in units of that variation. Phones LJ stretches a lot (vowels) absorb most of the change; phones she keeps steady (stops such as K) barely move. *Example*, "cry" at twice its natural length: scaling every phone equally gives K / R / AY ×2.0 each; the elastic fit ($z \approx 1.75$) gives ×1.75 / ×2.4 / ×2.0.

- **Stretch cap.** A word is stretched to at most 2.5× its natural length; any extra time becomes a pause after it, and the next word still starts on time. This happens for 148 words in 128 of the 1,248 test sentences.
- **Exact length.** The phone lengths are rounded so that their running total, not each length separately, is rounded; every word boundary is then within half a frame of the plan, and the output is padded or trimmed to exactly the reference's number of samples. The length error is checked on every call, so it is 0 by construction.
- **Diphthongs are split only inside the model.** The main system splits each diphthong into two tokens just before the model and adds the two predicted lengths back together afterwards; the lexicon, the aligner and the timing fit never see the extra tokens.

4. How the systems are evaluated

Requirement	Metric	Definition
Exact length	Length error	output length − reference length, in samples; must be 0
Word timing	Word-boundary error (MAE); share within 20 / 50 ms	MAE = mean absolute error: for every word, how many ms its start and its end in the output are from the **hand-annotated** start and end in the reference (output times found by MFA), averaged
Pauses	Pause F1	pause = a silence of at least 100 ms between two words. F1 combines precision (output pauses that are real) and recall (real pauses the output has); 1 = perfect
Exact words	WER, CER	word / character error rate: the share of words / characters wrong when Whisper large-v3, OpenAI's speech recognizer, transcribes the output; text and transcript are normalized the same way
US accent	US-accent probability	from CommonAccent ECAPA, a neural accent classifier; 0 to 1
LJ voice	Speaker similarity	cosine similarity (−1 to 1) of voice embeddings from an ECAPA speaker model: to LJ (should be high), to the reference speaker (should be low)
Naturalness	UTMOS	an automatic predictor of the mean opinion score (MOS) listeners would give, 1 to 5
Robustness	Stretch ratio	output speech length ÷ the model's natural length for the same words (1.0 = LJ's own pace)

Table 6. Metrics. All evaluation models are pinned to exact versions, checked by file fingerprint (sha256), and loaded offline.

- **Statistics.** Averages come with 95% confidence intervals (CI): ranges that contain the true average with 95% probability, computed by bootstrap (re-drawing the sentences at random 1,000 times). Systems are compared *paired*: both speak the same sentences, so the difference is taken sentence by sentence. Each comparison uses the Wilcoxon signed-rank test, which asks whether these differences are centred on zero. Its p-value is the probability of a difference at least this large if the systems were really equal. Because 8 metrics are tested at once, some would look significant by chance; the Holm correction raises the p-values to account for that, and a difference is called significant only when the corrected p is below 0.05. Comparisons are made overall, per accent and per tercile (three equal groups) of how much the reference stretches LJ's pace.
- **Decision rules written before results.** The tuning rule and the success criteria of each experiment were written in the decision log before the experiment ran.
- **Extra measurements.** *Duration fidelity*: where a word ends up in the output against where the pipeline placed it; this isolates the model's own timing error from the error of aligning the reference. *Aligner calibration*: the same measurement for both aligners on real recordings. *Sound change within phones* (§3.4). *Formant probe*: the resonance frequencies F1 and F2 that identify a vowel, measured with linear predictive coding (LPC), to see whether a diphthong glides. *Pace ρ (rho)*: a reference word's length divided by LJ's typical length for the same phones.

Caveats, stated rather than hidden.

- The accent classifier's US probability is close to 0 or 1 for single sentences and can flip on changes nobody hears, so it is only read as an average over many sentences.
- MFA judges systems that were trained on MFA durations, which may favour them. The second judge, the CTC aligner, places word starts +38 ms late on real speech (MFA: +0 ms), so CTC numbers are only read against that offset, never as a ranking.
- UTMOS chose the settings of the audio-stretching tool used for augmentation (on LJSpeech only) and is also an evaluation metric.
- M1's development-accent numbers are slightly optimistic, because 120 of those 1,200 references were used for tuning. The test numbers have no such bias.

5. Experiments and ablations

Each later experiment started from a listening observation. The input was checked first; the defect was measured with a targeted measurement; its cause in the design was found; a minimal fix was tested against a control that differs in that one setting; and the same measurement plus all metrics were repeated. All decisions here were made on the development accents; an ablation is an experiment that adds or removes one component to measure its effect.

5.1 Tuning the synthesis settings (development accents)

21 settings were tried on a fixed set of 120 development references: how the timing is transferred (the elastic fit, scaling every phone equally, or copying the reference's phone lengths), the stretch cap, the noise scale (how much randomness is sampled from the VITS prior; more is livelier but less clear) and the aligner. The rule, fixed before running: among elastic settings within 5 ms of the best timing error, take the one with the highest UTMOS. Chosen: elastic fit, cap 2.5, noise scale 0.333, MFA (21.9 ms; best 21.5 ms; UTMOS 4.21; word error rate 2.5%).

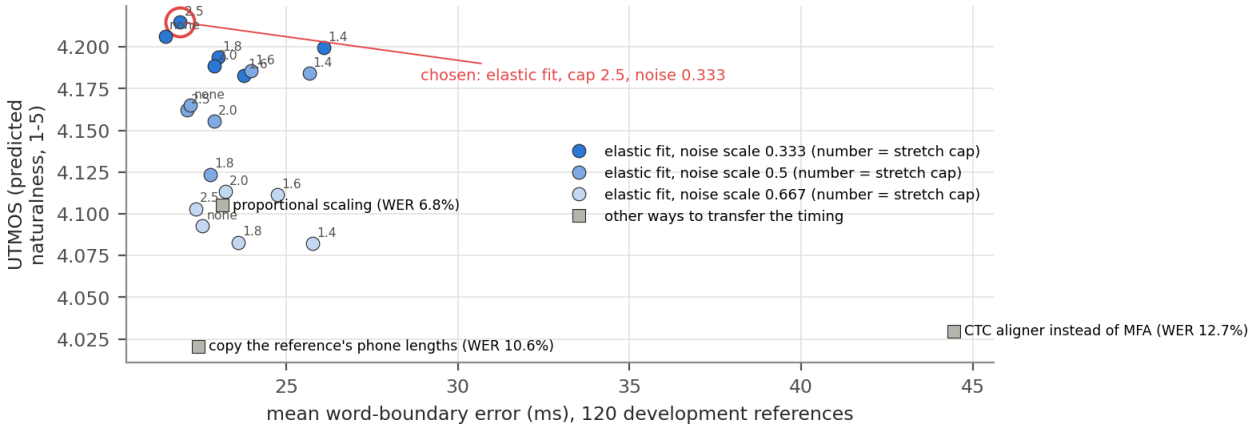


Figure 4. The tuning grid. Each dot is one setting. The noise scale drives the word error rate (0.333: 2.5-3.5%; 0.667: 4.6-6.7%); a looser stretch cap lowers the timing error at no cost in naturalness; copying the reference's phone lengths gives by far the worst word error rate, because the reference's phone timing carries its accent. So only word boundaries are copied.

5.2 First comparison: M1 against M0 (development accents)

1,200 development references	M0 (FastSpeech2)	M1 (VITS-MFA)
Word-boundary error	31.3 ms	**23.7 ms** (p ≈ 1e-140)
Word boundaries within 20 ms	55.4%	**68.5%**
Pause F1	0.75	**0.84**
Word error rate	10.0%	**3.5%**
UTMOS	3.94	**4.20**
US-accent probability	0.73	0.72 (not significant)

Table 7. Both systems receive the same durations; M1 renders them more precisely. p ≈ 1e-140: the chance that the difference is accidental is about 10^{-140}. US-accent probability did not improve, which set up the next experiments.

5.3 "Words end sharply" → position-in-phone embedding

Measurements ruled out loudness, pitch and the duration predictor at word ends. What differed was the sound change *inside* phones: 14% less than LJ, 20% less in the middle third. The cause: without blank tokens, every frame of a phone has the same target. The fix gives each frame a learned vector for its position in the phone (frame k of a d-frame phone goes to step ⌊16(k + 0.5)/d⌋, from 0 at the start to 15 at the end). **Result against m1_cont** (the same 20,000 steps without the change): the sound change inside phones did not move (0.856× vs 0.858× LJ; the embedding's effect sits at the edges of phones, which this measurement leaves out), but the US-accent probability rose from 0.680 to 0.769 (p = 5e-31; higher in all three accents), with no significant change elsewhere. m1_pos became the base for the next experiments.

5.4 Pace augmentation, Ablations A and B (negative result)

Hypothesis: training on LJSpeech clips that were made slower or faster teaches the model to stay natural when a reference forces unusual word lengths. **A (random paces):** each copy gets a pace drawn uniformly between −1.5 and +1.5 (in the units of z, §3.5), each word a small random deviation from it, and some pauses are lengthened or inserted with recorded room noise. **B (realistic paces):** word paces and pauses are drawn from the statistics of the development-accent speakers (each word's pace correlated with the previous word's; a pause after 22% of words); only word lengths are used, never their phone timing. Two copies of every clip, mixed 60/40 with the original clips; the duration loss ignores the copies; the 5% of copies that lost the most naturalness were removed.

A change forced by measurement: the planned audio-stretching tool, Rubber Band R3, damaged every clip (UTMOS 4.38 → 2.92, about 1.2 points even when a clip changed length by less than 5%). It was replaced with WSOLA (waveform-similarity overlap-add, which changes speed without changing pitch), written for this project; the average UTMOS loss fell from 1.34 to 0.32, and re-aligning the stretched clips found word boundaries a median of about 7 ms from where they were planned.

1,200 development references, compared with m1_pos	m1_pos	abl_a_pos	abl_b_pos
Word-boundary error	24.16 ms	24.04 ms (n.s.)	24.21 ms (n.s.)
Word error rate	3.9%	3.4% (n.s.)	3.9% (n.s.)
US-accent probability	0.769	**0.642** (p < 1e-10)	0.747 (n.s.)
UTMOS	4.18	**4.07** (p < 1e-10)	**4.10** (p < 1e-5)

Table 8. n.s. = not significant after Holm correction. Neither ablation improved timing or word error rate, even for the most stretched third of the sentences; both cost naturalness, and B shows no gain even on the accents its statistics came from. The likely cause: 40% of each batch carried leftover stretching artifacts, which the model learned. A's paces are far wider than real speech, which explains its loss of US-accent probability.

5.5 "cry" heard as "cra" → two-token diphthong

The lexicon gives "cry" → K R AY1 (correct), so the error was the model's. A diphthong is a vowel that glides, like the AY of "cry" moving from /a/ to /ɪ/. Measuring the formants (F1 and F2, the resonances that identify a vowel) showed the reference gliding (F2 up 285 Hz, F1 from 737 to 554 Hz) while M1, m1_cont and m1_pos held a flat /a/; across LJ's validation clips every model flattened diphthongs (average F2 movement 307-321 Hz against 396 Hz for LJ). **Cause: the token design.** The original VITS read "cry" as the IPA characters k, ɹ, a, ɪ with blanks between them, so a diphthong had two targets and a transition. Our ARPAbet input made it one token with one target. **Fix:** each diphthong (AW, AY, EY, OW, OY, at all stress levels) becomes two tokens, X and X+ (72 → 87 tokens). MFA gives one length for the diphthong, which is split evenly between the two; for the timing fit the two predicted lengths are added back together; X+ starts as a copy of X, so the model starts as m1_pos.

1,200 development references, compared with m1_pos	m1_pos	m1_pos_diph	p (Holm)
Word-boundary error	24.16 ms	**23.48 ms**	< 1e-6
Word error rate	3.9%	**2.7%**	< 1e-6
Character error rate	2.0%	**1.2%**	< 1e-6
UTMOS	4.18	**4.20**	2e-4
US-accent probability	**0.769**	0.738	1e-4
Average F2 movement in diphthongs (LJ: 396 Hz)	321 Hz	**389 Hz**	

Table 9. "cry" now glides (F1 864 → 528 Hz, F2 up 215 Hz). The one criterion not met, the US-accent probability, was investigated: its drop does not follow the number of diphthongs in a sentence (Spearman correlation −0.03, i.e. unrelated), sentences without any diphthong dropped too, the classifier flips both ways (91 large drops, 58 large rises), and the four largest drops sounded identical to the listener. Decision (D-85): m1_pos_diph is the main system. The code was then frozen and the test accents opened once.

5.6 The immediate baseline V0, and how faithfully each model renders its durations

V0 is the same pretrained VITS before MAS was replaced, given the same word timing through the same pipeline. To see *why* it misses the timing, the duration-fidelity measurement compares where MFA finds each word in the output with where the pipeline placed it, which removes the error of aligning the reference.

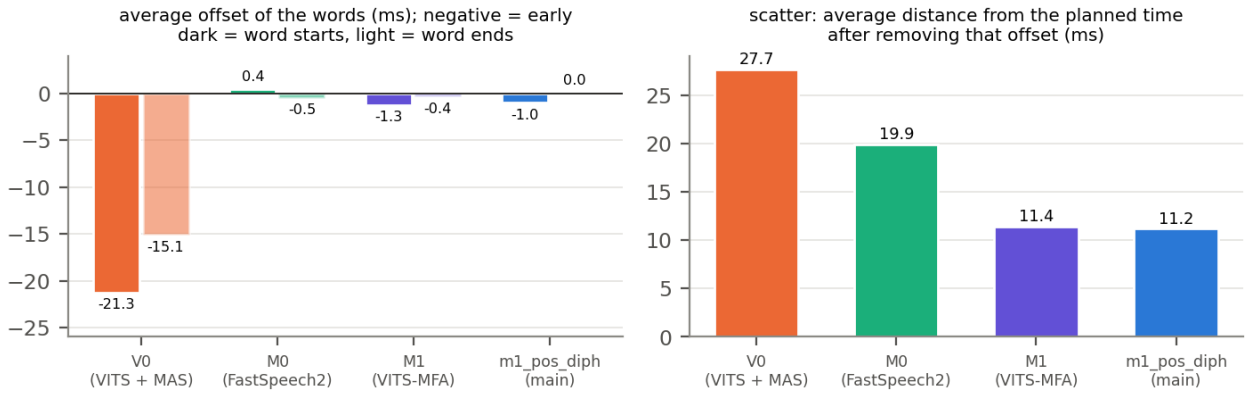


Figure 5. Duration fidelity on the test accents. Left: the average offset of word starts and ends from where they were placed; V0's words come out about 20 ms early. Right: the remaining scatter once that offset is removed; V0 is also the least precise. The fine-tuned systems put words almost exactly where they are told. The CTC aligner, an independent second judge measured against its own offset on real speech, finds the same early shift for V0.

The two aligners agree on the direction and size of V0's shift; the CTC aligner is too imprecise to rank the scatter, so that part rests on MFA. **The trade-off, stated plainly:** V0 is the better system on word error rate and UTMOS. Fine-tuning on MFA durations without blank tokens bought controllable timing and a more US-like accent, at a cost that the two-token diphthong already halved (word error rate above V0's: M1 +1.7 percentage points, main system +0.8).

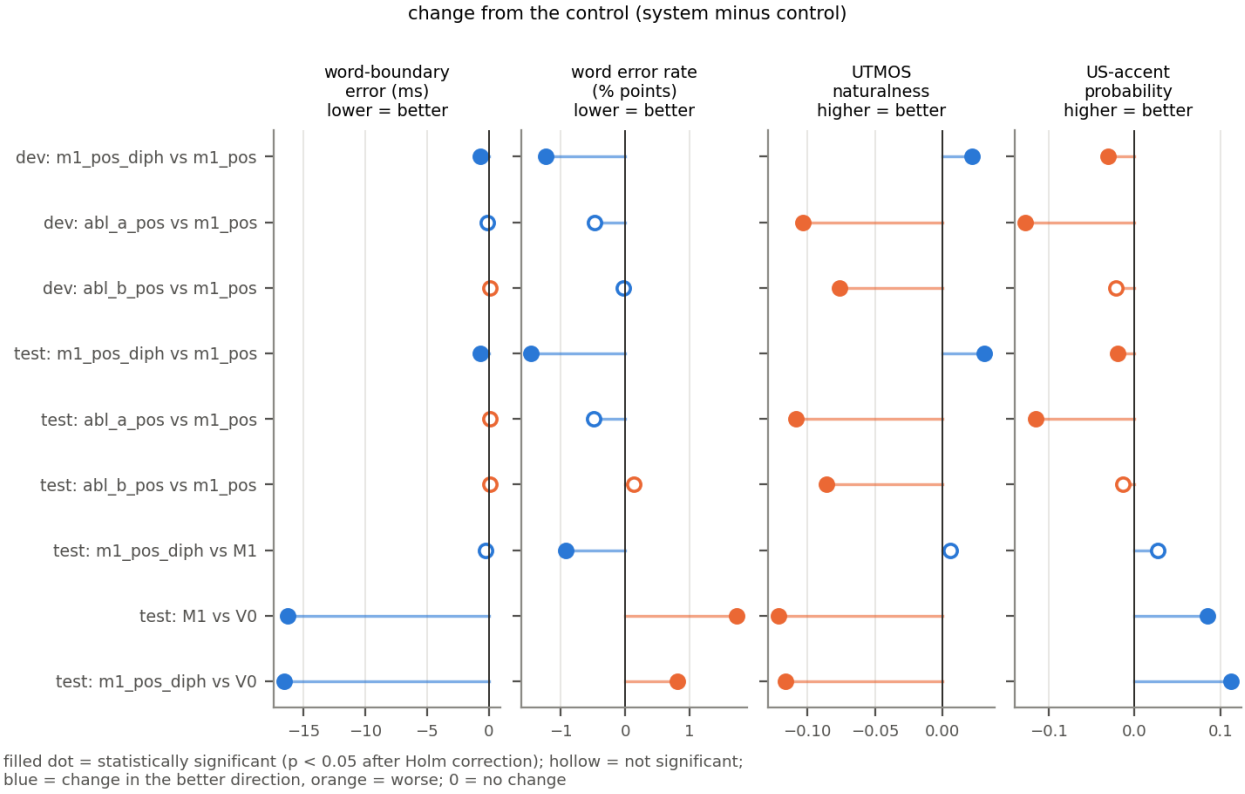


Figure 6. Every comparison between a system and its control, as the average per-sentence difference (system minus control). "dev" = development accents, "test" = test accents. A Holm-corrected p-value accounts for the 8 metrics tested in each comparison (glossary); filled dots are differences that are very unlikely to be chance.

6. Results on the test accents

Arabic, Korean and Vietnamese: 12 speakers, 1,248 references, evaluated once after the code was frozen. MFA aligned all 1,248 references (the fallback aligner was never needed). The length error is 0 for every output of every system.

System	Word-boundary error, ms [95% CI]	Within 20 ms	Pause F1	Word error rate	US accent probability	UTMOS (1-5)	Similarity to LJ voice
Non-US references (input)				6.3%	0.486		
LJSpeech recordings				1.9%		4.36	
V0 (VITS + MAS)	40.5 [39.7, 41.2]	41.2%	0.815	2.1%	0.649	4.31	0.802
M0 (FastSpeech2)	31.9 [31.1, 32.7]	55.3%	0.726	9.3%	0.746	3.96	0.766
M1 (VITS-MFA)	24.2 [23.5, 24.9]	69.3%	0.822	3.8%	0.734	4.19	0.805
m1_pos	24.6 [24.0, 25.4]	68.3%	0.826	4.4%	0.781	4.17	0.806
m1_pos_diph (main)	**23.9 [23.3, 24.7]**	**69.6%**	**0.835**	**2.9%**	**0.762**	**4.20**	**0.800**
abl_a_pos	24.7 [24.0, 25.5]	68.3%	0.825	3.9%	0.665	4.06	0.801
abl_b_pos	24.7 [24.1, 25.4]	68.3%	0.826	4.5%	0.767	4.08	0.796

Table 10. Main results (results/<system>/test/summary.json). CI = confidence interval. Similarity to the reference speaker's voice stays low (0.11 for the main system): the reference's voice does not carry over.

Compared pair (test)	Word-boundary error (ms)	Within 20 ms (% points)	Word error rate (% points)	UTMOS	US accent probability
m1_pos_diph vs m1_pos	-0.7 (p 1e-8)	+1.2 (p 1e-4)	-1.5 (p 1e-8)	+0.03 (p 1e-5)	-0.019 (p 0.017)
abl_a_pos vs m1_pos	+0.0 (n.s.)	-0.1 (n.s.)	-0.5 (n.s.)	-0.11 (p 4e-38)	-0.116 (p 8e-45)
abl_b_pos vs m1_pos	+0.1 (n.s.)	-0.1 (n.s.)	+0.1 (n.s.)	-0.09 (p 4e-23)	-0.014 (n.s.)
m1_pos_diph vs M1	-0.3 (n.s.)	+0.3 (n.s.)	-0.9 (p 2e-5)	+0.01 (n.s.)	+0.027 (n.s.)
M1 vs V0	-16.3 (p 4e-189)	+28.1 (p 2e-184)	+1.7 (p 3e-11)	-0.12 (p 8e-33)	+0.086 (p 3e-12)
m1_pos_diph vs V0	-16.5 (p 5e-192)	+28.4 (p 1e-185)	+0.8 (p 0.004)	-0.12 (p 3e-28)	+0.113 (p 8e-20)

Table 11. Average change per sentence, first system minus second, on the 1,248 test references. p = Holm-corrected p-value of the Wilcoxon signed-rank test; n.s. = not significant (corrected p of 0.05 or more). Points = percentage points.

Findings.

- **Replacing MAS with MFA durations is what makes the output follow the reference timing:** main system against V0, −16.5 ms word-boundary error and +28 percentage points of boundaries within 20 ms, in every accent and at every amount of stretching; the cost is +0.8 points of word error rate and −0.12 UTMOS.
- **VITS-MFA beats FastSpeech2 on everything but accent** (M1 against M0: −8 ms timing error, word error rate 9.3% → 3.8%, UTMOS +0.23).
- **The two-token diphthong also helps on the test accents**, against its control: word error rate −1.5 points, timing error −0.7 ms, UTMOS +0.03.
- **Pace augmentation is a negative result on the test accents too:** no gain in timing or word error rate; UTMOS −0.11 (A), −0.09 (B).
- **The development conclusions held on the unseen accents**, whose pace was similar (pace figure at the end of this section).

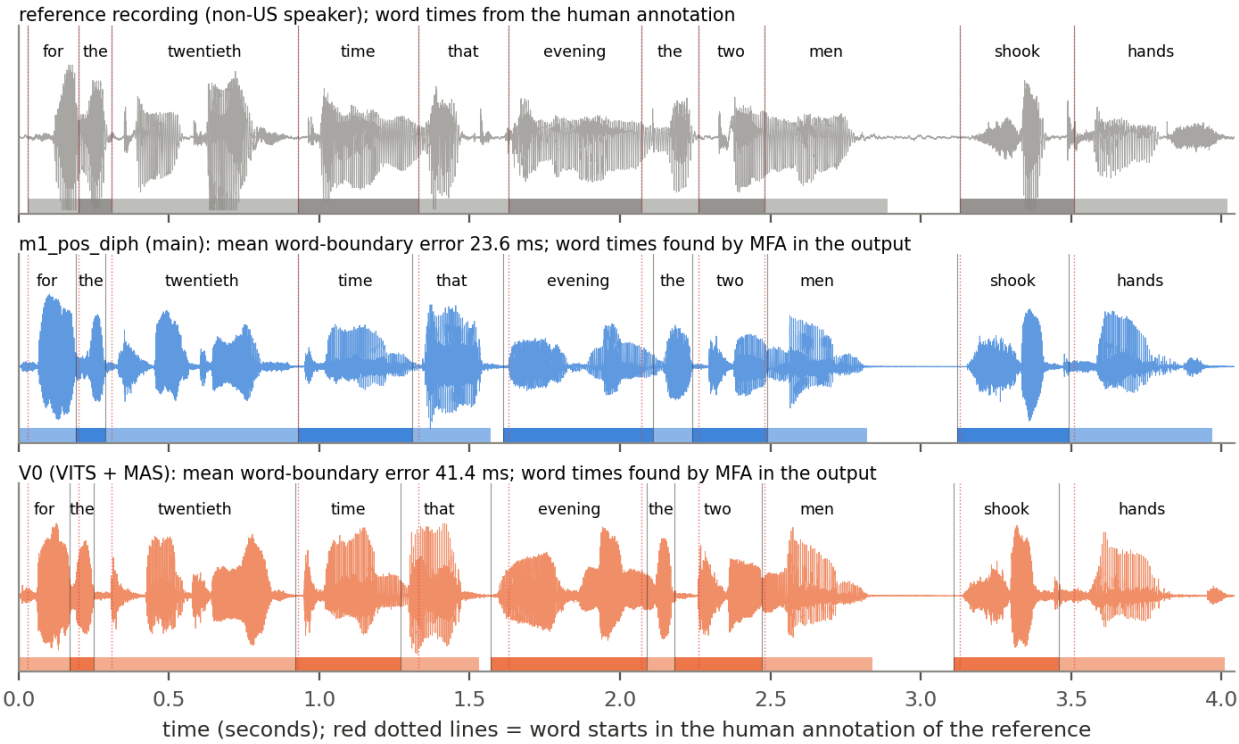


Figure 7. The headline sample, YBAA_arctic_a0003 ("For the twentieth time that evening the two men shook hands"). Top: the non-US reference with the hand-annotated word times. Below: the main system and V0, each with the word times MFA finds in the output. All three have the same length by construction; the main system's words sit on the reference's word starts, V0's drift early. Audio: samples/demo/arabic/YBAA_arctic_a0003/.

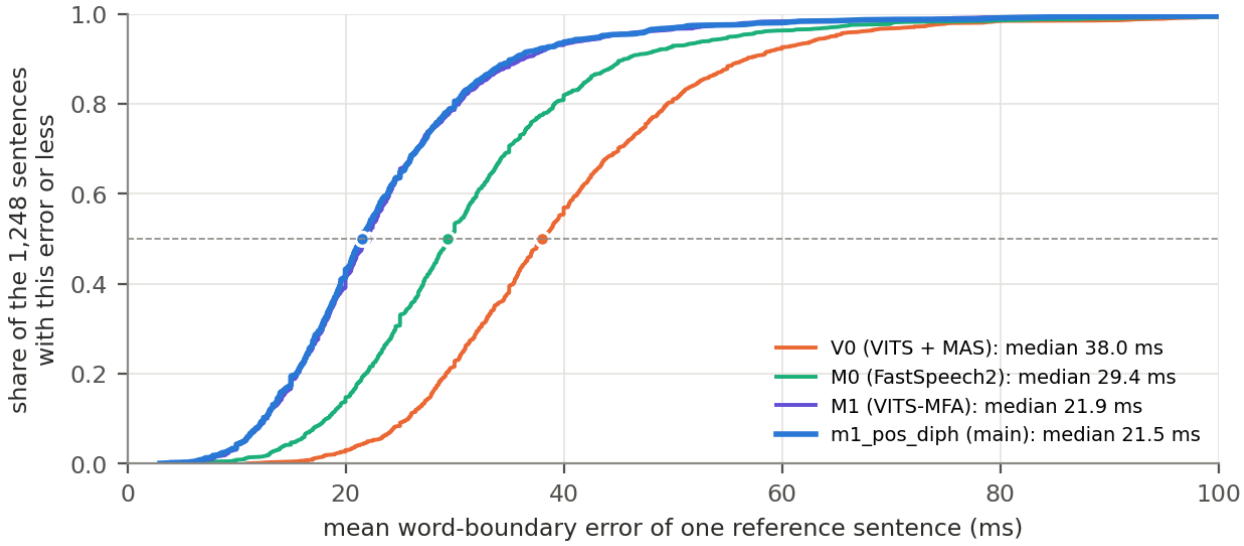


Figure 8. How the error is distributed over the test sentences: for each error value on the horizontal axis, the share of sentences with that error or less (a cumulative distribution). A curve further left is better. The whole distribution of the main system is shifted left of V0 and M0, not only its average. 90th percentiles (90% of sentences are below): V0 56.8 ms, M0 45.6 ms, M1 36.3 ms, m1_pos_diph 35.5 ms.

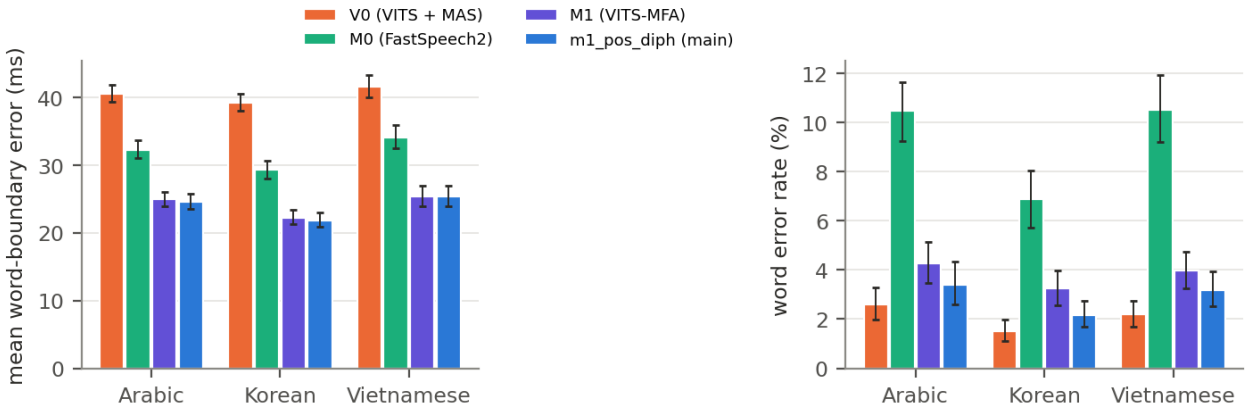


Figure 9. Results per test accent. Error bars: 95% confidence intervals. The timing gain over V0 and M0 holds in every accent; Korean speakers are the easiest for every system.

Accent	US-accent prob. of the references	V0: timing error / word error rate	M1: timing error / word error rate	m1_pos_diph: timing error / word error rate	Main system: US-accent prob.
Arabic	0.455	40.5 ms / 2.6%	24.9 ms / 4.3%	24.6 ms / 3.4%	0.721
Korean	0.595	39.2 ms / 1.5%	22.2 ms / 3.3%	21.8 ms / 2.2%	0.793
Vietnamese	0.406	41.6 ms / 2.2%	25.4 ms / 4.0%	25.4 ms / 3.2%	0.771

Table 12. Per test accent. The US-accent probability rises from 0.41-0.60 for the references to 0.72-0.79 for the outputs.

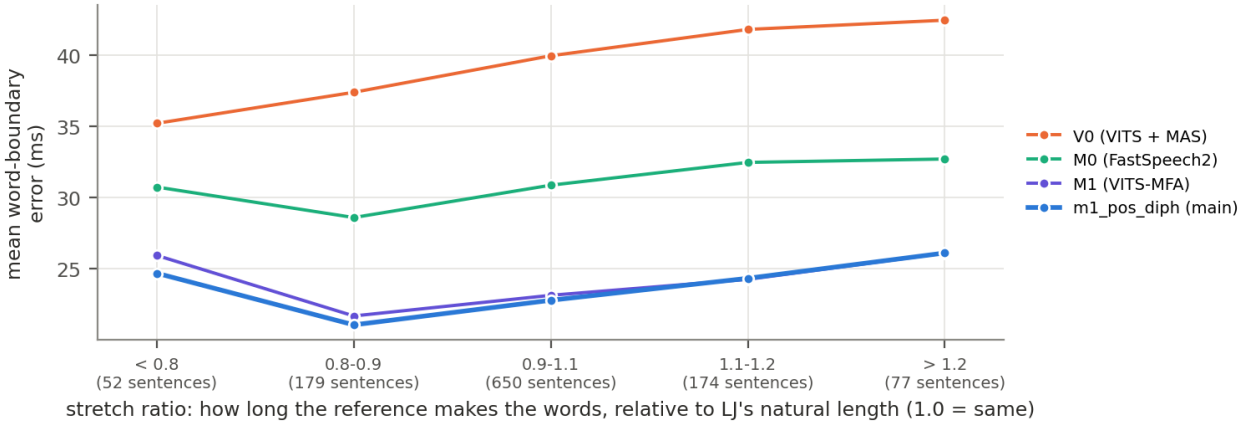


Figure 10. Timing error against the stretch ratio (how much longer or shorter the reference makes the words than LJ would say them; sentences grouped by the main system's ratio, available for 1,132 of them). Every system's error rises when the reference is much slower or faster than LJ; the gap to V0 holds in every group.

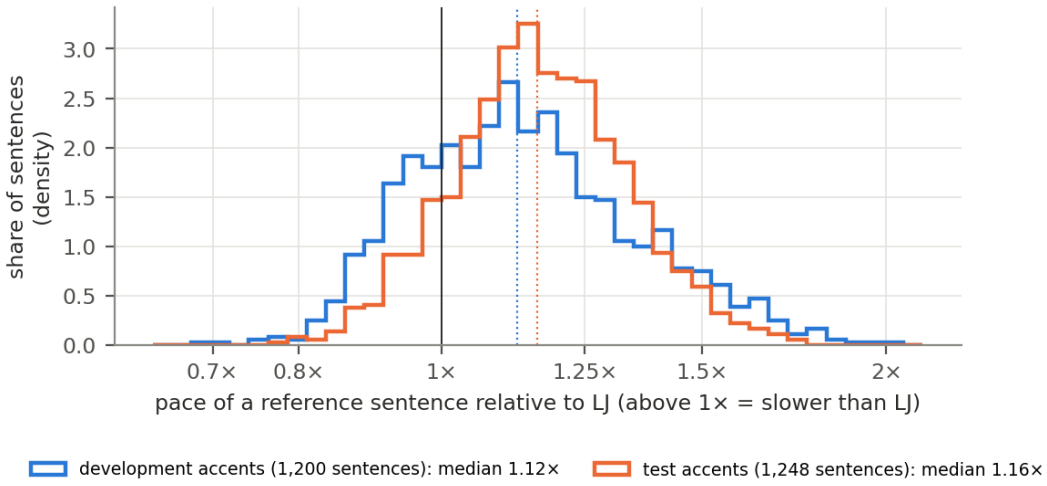


Figure 11. Pace of the reference sentences relative to LJ (ρ, rho: each word's length divided by LJ's typical length for the same phones, averaged per sentence). Both accent groups are about 12-16% slower than LJ with a similar spread, so the development accents were a fair stand-in for the test accents.

Failures are shown, not hidden. The worst sentence per accent is in samples/demo/ (e.g. THV_arctic_a0067: 148 ms for the main system and V0 alike). The second aligner (CTC) measured on the outputs gives 45.6-52.0 ms for all systems, but because it places word starts 38 ms late on real speech, V0's early words partly cancel that offset; its numbers are therefore read only against its own offset (§5.6), never as a ranking.

7. Efficiency

Quantity	Value (main system)
Parameters used at synthesis	28.1 million: waveform decoder 14.3 M, flow 7.1 M, text encoder 6.3 M, duration predictor 0.35 M
Parameters used only in training	54.0 million: discriminators 46.7 M, posterior encoder 7.2 M
Computation	27.2 billion multiply-accumulate operations (GMACs) per second of audio; 99.7% of it in the flow and decoder. Checked by hand for one layer: 7 × 192 × 512 × 86 = 59.2 million
Real-time factor (computing time ÷ audio length), laptop GPU	model only 0.012 (about 83× faster than real time); end to end 0.90, dominated by aligning the reference
Training speed (L40S GPU)	0.229 s per step of 32 clips; 114,000 steps ≈ 7.3 hours

Table 13. Measured with results/profile_*.json: 10 warm-up runs and 50 timed runs (the aligner: 1 + 5), median times, synchronized with the GPU. Operations counted with ptflops, a library that counts the operations of a network.

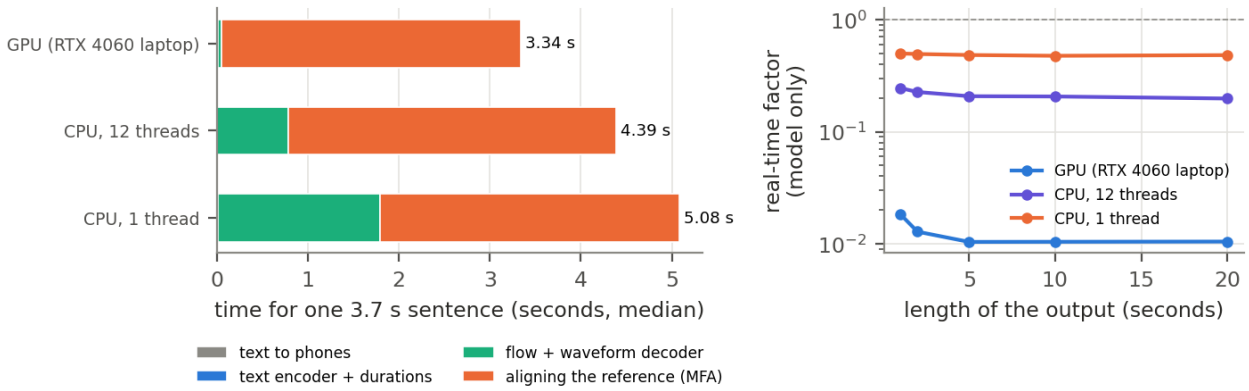


Figure 12. Left: time to produce one 3.7-second sentence, split by stage, on the laptop GPU (NVIDIA RTX 4060) and on the CPU with 12 or 1 threads (processor cores). Aligning the reference with MFA takes over 3 seconds everywhere and dominates; on the GPU the model itself needs 45 ms. Right: the model's real-time factor for outputs of different lengths (logarithmic scale; below the dashed line = faster than real time).

8. Robustness and live testing

- **Difficult-input suite** (21 cases, all pass on GPU and CPU): digits, currency, dates, abbreviations, an unknown name; MP3 files, 8,000 and 48,000 samples per second, stereo, clipped, noisy and echoing audio; recordings over 20 s, a 1.8 s pause, 1.6× speed, a word cut off; a word in the text that is not spoken, a false start, and a forced use of the fallback aligner. Pass = exact length, or a clear fallback.
- **Typed text:** 121 test inputs (119 right; the two misses are guessed pronunciations of unknown names) and 12 end-to-end rehearsal cases, including inputs that must be refused.
- **Unit tests:** 317 pass on the author's machine; 309 pass and 9 are skipped on a fresh copy without the datasets.
- **Browser demo:** a visitor records or uploads a sentence and hears all seven systems side by side; 50 recorded sessions are kept in samples/browser_tests/.

Live testing of the browser demo found failures that read-aloud corpora never contain:

Input	What happened	Cause	Status
abc, chaganti (words first seen at synthesis)	MFA marked them unknown; the demo used the less accurate fallback aligner	MFA reused an old copy of the lexicon it had cached	**Fixed:** the cache is now tied to the lexicon's fingerprint
bcd, xyz	pronounced as an invented word (bcd → "bed")	nothing marked them as letters	**Fixed:** unknown lowercase strings of 3-5 letters without a vowel are spelled out
the letter "a" in a, b, c	aligned to only 30-80 ms; the output says "uh"	one pronunciation for "a": AH0, the article	**Open:** needs the letter-name reading (EY1 gave 220-330 ms)
"I" in may I know you	"I" got 30 ms, the aligner's minimum	two vowels meet (EY then AY) with no consonant between	**Open:** fine after a pause or at the start
ijk; brrr, grr; pq	invented word; wrongly spelled out; not spelled	limits of the hand-written rule	**Open:** needs a learned text normalizer

Table 14. Live-testing findings (E-35). The test results are unaffected: every evaluated word was in the lexicon before MFA first cached it.

9. Limitations and future work

- **The accent classifier is a weak judge;** a listening test (for example MUSHRA, where listeners rate versions side by side) with accent ratings is the right next measurement.
- **Part of the accent lies in timing:** forcing LJ's voice onto a non-US speaker's word timing limits how American it can sound.
- **A fixed alignment flattens phones** (14-16% less sound change inside phones); V0 keeps a better word error rate and UTMOS.
- **No word-boundary marker** in the model's input ("a name" and "an aim" are the same phones); the diphthong is always split 50/50.
- **The ablations started late** (20,000 steps from M1), so they test late-stage augmentation only.
- **V0 needs an adapter** (lexicon phones written as IPA, a rule for blank tokens); a learned adapter might narrow its gap.
- **End-to-end speed is limited by the aligner** (3.3 s per reference against 45 ms for the model).

Future work, most promising first: (1) let the model learn its alignment again (MAS) but only *inside* each word's span from MFA, restoring the feedback while keeping timing control; (2) mark word boundaries in the model's input; (3) learn where to split each diphthong; (4) split other gliding sounds (CH, JH, ER) the same way; (5) a listening test as the accent judge; (6) a faster, always-loaded aligner for live use; (7) letter names and vowel-only words for live input.

Takeaway. Make durations an input, copy the timing of words (not of phones), and check how sounds are tokenized before adding data: replacing MAS cut the word-timing error from 40.5 to 24 ms, and the largest later gain came from giving a diphthong two tokens, not from more training data.

10. Reproducibility and deliverables

Step	Command	Result
Software environments	make env	isolated, version-pinned conda environments tts and mfa (exact lists in envs/)
Trained models	make models	the frozen models, downloaded from the Hugging Face Hub at a fixed revision and checked by file fingerprint (sha256)
Synthesize	make infer, make live-demo	output audio of exact length, word timings, a timing plot
All test outputs	make outputs	the 1,248 evaluated outputs of every system
Tests	make test, make text-stress, make rehearse	309 unit tests pass on a fresh copy; 119 of 121 typed inputs; 12 of 12 rehearsal cases
Browser demo	make web-demo (and make web-tunnel to share it)	record or upload, all seven systems side by side
This report	make report-pdf	docs/meeami-tts_report.pdf, from the result files in the repository

Table 15. Checked from a fresh copy of the repository with no login and an empty download cache (D-104).

Deliverable	Where
Code (frozen at git tag freeze-v1)	https://github.com/rrtchaganti/meeami-tts
Detailed report linking every claim to its code	docs/REPORT.md; decision and error log docs/DECISIONS.md
Audio evidence: reference, hand annotation, all 7 outputs, word timings, 3 accents	samples/demo/ (21 samples; each sample's timing error recomputed and checked against the evaluation, 147 of 147 match)
Listening sets behind the experiment decisions; browser recordings	samples/listening/, samples/browser_tests/
Trained models and every test output	https://huggingface.co/Raviteja546/meeami-tts

Table 16. Where each deliverable is.

Appendix A. Deviations from the plan

The plan said	Done instead	Why
Rubber Band R3 to stretch audio	WSOLA (written for this project), same stretch plan	R3 cost about 1.2-1.5 UTMOS on every clip; WSOLA 0.32
torchaudio's forced-alignment function	the same algorithm written in numpy, same MMS model	the installed torchaudio package lacks that function
Unknown words from g2p_en	plus a rule for possessives	g2p_en mangled 104 possessives
Three GPU machines	one L40S: M1, then 20,000-step continuations	availability
Speed measured on the data-center GPU	laptop GPU and CPU only	could not run on the rented machine
72 model tokens	87 for m1_pos_diph only	diphthong flattening was diagnosed
UTMOS loaded with its default loader	weights pinned by fingerprint and loaded safely (weights only)	the default loader ran unverified code
Baseline M0 only	plus V0, the original VITS with MAS (added after the freeze, synthesis only)	the immediate baseline for replacing MAS
Text normalizer as specified	plus the typed-text rules (after the freeze)	live input; corpus words unchanged (tested)
MFA's single-file aligner with a fixed cache	cache tied to the lexicon's fingerprint	an old cached lexicon made new words unknown

Appendix B. Claim → evidence

Claim	Evidence (files in the repository)
Output length exact for every output	length_error = 0 in every results/*/test/per_utt.csv
MFA's error on accented speech ≈ 19 ms	results/mfa_l2_dev.json, reports/mfa_l2_dev.md
Replacing MAS makes timing follow the reference	results/paired/test_m1_pos_diph_vs_v0.csv; samples/demo/
V0 renders durations early and scattered	results/fidelity/test.json, calibration_dev.json
A fixed alignment flattens phones	results/phone_dynamics/*.json
Cap 2.5 and noise scale 0.333 chosen by a rule fixed in advance	results/tune/grid.json, choice.json
The ablations give no gain	results/paired/{dev,test}_abl_*_vs_m1_pos.csv
The two-token diphthong improves timing, word error rate and UTMOS; "cry" glides	results/paired/*_m1_pos_diph_vs_m1_pos.csv; samples/listening/p2c_diphthongs/
Training curves; reference pace	results/report_data/m1_training_curves.csv, pace_rho.csv
The test accents were not read before the freeze	git tag freeze-v1 → commit 5889ecf; tts/guard.py
The hosted models are the frozen ones	MANIFEST.sha256 on the Hugging Face Hub; scripts/export_infer.py --verify